

Reproducibility with R

Thomas Sol Dourdin

2026-09-30

Table of contents

1	Introduction	4
2	Project architecture	7
2.1	The fundamental structure	7
2.2	Additional elements	9
3	Project implementation - the <i>targets</i> package	10
4	Project implementation - the <i>renv</i> package for a transportable project-specific library	15
5	Making your project fully reproducible with the power of images & containers - <i>Docker</i>	17
5.1	What images & containers are?	17
5.2	Creating a <i>Docker</i> image - the Docker file	18
5.3	Looking at the <i>Docker</i> image, run it and retrieve the results	21
5.4	Sharing your project as <i>Docker</i> image	22
6	Project versioning - Git and Github	23
6.1	What are we talking about?	23
6.2	Installing Git	23
6.3	Configuring Git	24
6.4	Connecting Git and GitHub	25
6.5	Example for routine use	27
6.6	Connect RStudio to Git and GitHub	29
7	Rendering the work	31
7.1	Introduction to Rmarkdown	31
7.2	Structure of a Rmarkdown document	31
8	Summary: common reproducibility anti-patterns (and how to avoid them)	34
9	The reproducibility: a way to find help!	36
9.1	<i>styler</i> for a perfect formatting	36
9.2	<i>tribble</i> to present your data	37
9.3	<i>reprex</i> : exporting a fully reproducible script	39

Foreword: You will find in the following the presentation of diverse tools to enhance the reproducibility of your analytical work carried out with R, using the IDE RStudio. Consequently, you need to have both installed on your machine to properly follow and experiment the content. To do so, go to the [Posit](#) website and (i) install R (choose the version whether you have a windows or macOS system), (ii) install RStudio (choose the version whether you have a windows or macOS system).

1 Introduction

Modern science requires for each experiment to be **replicable**, i.e. the capacity to “*obtain[] consistent results across studies aimed at answering the same scientific question, each of which has obtained its own data*”. (National Academies of Sciences, Engineering and Medicine, 2019 dans [Hicks, 2023](#)). This acception of replicability refers to the assumption of **universality of science**. In other words, if we demonstrate the neurotoxicity of mercury on *Sepia officinalis* on 12 January 2026 in La Rochelle, this *must be true* in Porto or Namur 10 years later. Consequently, replicability relies on the **qualitative consistency** of scientific results, as well as rigorous sharing of experimental set up.

Since the advent of computers and numerical analyses, the notion of **reproducibility** has been added to that of **replicability**. A synonym of computational reproducibility, **reproducibility** refers to the ability to “*obtain the same results from the same data by using the same computational steps, the same code, and the same analytical conditions*” (National Academies of Sciences, Engineering and Medicine, 2019 dans [Hicks, 2023](#)). It concerns the **quantitative consistency** of results and should be considered an **indispensable tool** for peer-review of outputs resulting from a research project (reports, articles, expert assessments, etc.).

While replicability involves a number of good practices (robust experimental designs, laboratory notebooks, enough replicates, randomization, etc.), reproducibility likewise relies on a set of tools and good habits to be adopted in numerical analysis within a research project.

Yes, but here is the catch!

For several years now, science in general - and environmental sciences in particular - have been suffering from a **reproducibility crisis** ([Ioannidis, 2005](#)). This crisis results in part from the weakening of established good practices in the relevant scientific fields ([Harris et al., 2014](#)), but also from the rise of computing and Big Data, which entail the use of often complex datasets and analytical tools that are evolving at a frenetic pace, handled by practitioners who are still too rarely trained in best practices for reproducibility.

This document aims to present these good practices and a few essential basic tools for ensuring the reproducibility of research work based on the use of bioinformatics tools. **The R programming language will serve as our guiding thread.**

These good practices can be structured around four main pillars:

- **Project architecture:** It refers to the way one organizes the working directory, which should be efficient (simplified paths, easily understandable file names) and standardized, in compliance with the **Research Compendium** convention.
- **Project implementation:** As surprising as it may seem, actions performed through graphical interfaces are often not reproducible! It is therefore imperative to favor interaction with the machine via the **command line**, even though this may require long and demanding training. Moreover, the use and creation of *scripts* must follow certain rules. Likewise, the sequence of actions performed on the data (i.e. **workflow**) must be automated to avoid any operator-induced error or variation. Finally, it is important to keep in mind that any action performed on a computer - whether a command-line operation or the use of a point-and-click interface - will produce results that are closely tied to the **system environment**, which must therefore be taken into account.

To address all of this, two main R packages are essential to know: *targets* and *renv*. More complex and holistic tools, such as *Docker* allow to secure the whole environment, that is your R and packages versions but also the operating system, system libraries, etc. This latter tools are probably too much for most of our research projects conducted with R. But they can be very useful if you want to ensure a full reproducibility in time, and for more complex projects such as bioinformatics projects.

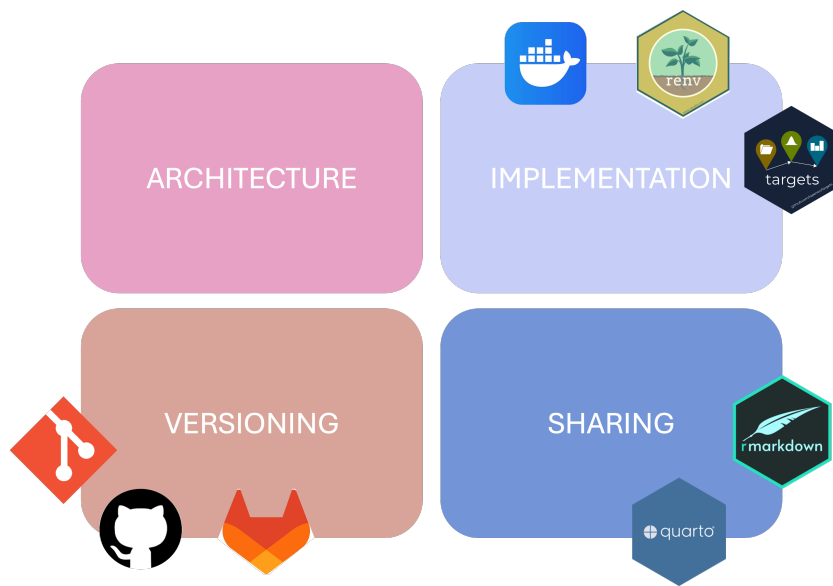
- **Project tracking:** Just as in the laboratory any modification to a protocol must be discussed with colleagues, collectively validated, and recorded in the lab notebook, any change made within a workflow must likewise be discussed, validated, documented, and **reversible**. To achieve this, **version control** (versioning) tools exist and should be used systematically.

Git (locally) and *GitHub* (remotely) are the version control tools most commonly used by the community.

- **Project sharing:** Project sharing should rely on tools that limit the manual integration of various objects into the manuscript, with the constant goal of minimizing the risk of individual error. These tools are based on the principle of **literate programming**, which consists of integrating tags directly within the text, whose visual rendering is only accessible after exporting the document to PDF (or Word, HTML, LaTeX, etc.).

In *RStudio*, the reference tool is *R Markdown*; it is *quarto* if you use the IDE new generation, i.e. *Positron*.

Before going further, keep in mind that reproducibility is not perfection. It is about making your work understandable, inspectable, and improvable. Consequently, Every little bit of extra practice is already a big step forward!



2 Project architecture

The project architecture must comply with the standards of the **Research Compendium** (Gentleman & Lang, 2012). It is a set of simple rules to follow in order to organize, in a **standardized** and **efficient** manner, the entire digital framework of your research project. By adopting this convention, anyone familiar with the Research Compendium will be able to quickly understand the project, extend it, disseminate it, and reproduce it. The Research Compendium thus constitutes the basic architecture of a reproducible project!

2.1 The fundamental structure

All elements of a research project must be contained within a single working directory, here *Reproducibility*. They must, however, be clearly separated within that working directory.



The minimum elements that must be present in your project's working directory are:

- **Data:** This folder should contain only the project's **raw data**, preferably set to **read-only**. Since the entire project relies on these data, they should never be manipulated ! Furthermore, cleaned data should already be considered as results and therefore should not be placed in this folder.
- **R:** This folder contains the project's **scripts**. These scripts hold the actions performed on the data. It is preferable to limit them to short scripts corresponding to specific tasks (cleaning, statistical analysis, graphical representation). As much as possible, **avoid endless scripts of hundreds or thousands of lines!** Moreover, it is important that each script is easily understandable by anyone. To achieve this, it is helpful to name

each script after the function in contains and number them in order of use. This allows the entire workflow to be easily visualized.

 01_Load_gasar.R
 02_Res_gasar.R
 03_Plot_length_gasar.R
 04_AnovaTest_gasar.R

To further facilitate understanding, it is also recommended to assign a standardized header to each script. There is no established standard; the header simply needs to allow reader to quickly understand which project the script belongs to, its role within the project, and, if necessary, how to contact the author.

```
#####  
# NAME_OF_PROJECT  
# NAME_OF_SCRIPT  
# OBJECTIF  
# CONTACT  
#####
```

- **Results:** This folder contains all the results: cleaned data, graphs, statistical outputs, etc. At a minimum, results should be named after the script that produced them. If multiple results are generated from the same script, they should be organized into subfolders named after their parent script. Unlike the **Data** folder, whose contents are sacred, and the **R** folder, which requires careful tracking, the contents of **Results** have no intrinsic value since they can be reproduced identically and *ad libitum*. Therefore, there is no need to fear manipulating results, deleting them, moving them, etc.
- **R_project:** Any research project that uses R as its programming language for analyses should be carried out within an **R_project**. Located at the root of the working directory, the .Rproj file associated to your project allows you to use **relative paths** for all files contained and produced in your working directory. Thus, by sharing your project along with the associated .Rproj file, it will never be necessary to modify the paths used in the scripts, nor specifying specific working directory. If you are not already systematically working within R_projects, now is the time to start! A tutorial for creating R_projects is available [here](#). This way, you will no longer need to specify your working directory environment using the `setwd()` function, which relies on absolute paths that are only valid on your own computer.

2.2 Additional elements

Some elements can be added to the working directory which, while not mandatory, are still welcome.

- **README.txt:** The **README** file is intended to explain the project. It is generally the first file opened when exploring a project. It can include all relevant information needed to understand the origin, context, objective, and execution details of the project. It is also the appropriate place to list all project-specific system information: operating system, R version, packages and their versions, etc.
- **Docs:** A **Docs** folder can optionally complement the working directory by containing, for example, documentation associated with specific packages.
- **Report:** This folder may contain all the files related to the reporting of your project, i.e. working reports, presentations, manuscripts, images... If you do the reporting with Rmarkdown or Quarto (see section 6), having this folder within the `R_project` working directory is required to facilitate the connection to your scripts and outputs.

💡 Preserving paths in any situations

Even if you work in a `R_project` properly structured and use only relative paths (i.e. `read.csv(Data/allo.csv)`), R markdown and Quarto will not find the root by themselves. Consequently, some inconsistencies will appear between the paths used in your scripts and the ones used by R markdown and Quarto. To avoid this, you should use the `here::here()` function.

`here::here()` forces R, R markdown and Quarto to search the path from the project's root. Concretely, `here()` goes up folders until finding a `.Rproj` file, a `.git/` or any other element specific to the root.

Therefore, rather specifying your path as `read.csv(Data/allo.csv)`, prefer to use `read.csv(here:here("Data", "Allo.csv"))`. By doing this, you will use relative paths in their most powerful form.

3 Project implementation - the *targets* package

targets is a package used for managing **workflows**. It will be your ideal ally to organize your project, develop it further, and also keep an eye on the **project architecture** - very useful when the project contains multiple datasets, numerous scripts, and a large number of results, especially when these are interdependent! It also allows you to visualize the update status of your various objects.

To understand how *targets* works, there is nothing better than a concrete example!

Note that it is recommended to write your scripts in the form of functions. This allows (i) limiting the number of objects created when the scripts run, and (ii) simplifying as much as possible the use of the *targets* management script.

For this example, we are going to use the dataset **Allo.csv** available [here](#). It contains allometrics measurements of two bivalve species *Crassotrea gasar* and *Anadara senilis* from several sampling stations in the Sine Saloum Delta, Senegal.

For this project, we want to briefly compare the shell length of *C. gasar* between the different sampling stations. We want to:

- (i) load the dataset and keep only *C. gasar* data;
- (ii) compute descriptive statistics for each sampling station;
- (iii) plot the data ;
- (iv) make a multiple comparison test (ANOVA)

These four steps correspond to the following functions:

```

# Loading and tidying dataset
#-----
load_gasar <- function(file){
  read_table(file=file, locale = locale(decimal_mark = ",")) %>%
    data.frame() %>%
    filter(Espece == "Crassotrea_gasar")
}
#-----

# Compute descriptive statistics
#-----
Res_gasar <- function(data) {
  data %>%
    group_by(Station, Lot) %>%
    summarise(length=mean(Longueurs) )
}
#-----

# Plot the data
#-----
Plot_length <- function(Resume){
  ggplot(data = Resume, aes(x=Station, y=length, color=Lot)) +
    geom_point()
}
#-----

# Run analysis of variance for multiple levels
#-----
AnovaTest_gasar <- function(data, a, b){

  Obj <- aov(a ~ b, data=data)
  Shap <- shapiro.test(Obj$residuals)
  if(Shap["p.value"] > 0.05){
    Lev <- leveneTest(Obj$residuals, data$Station)
  } else(return("No residual normality"))
  if(Lev["group","Pr(>F)"] > 0.05){
    Res <- anova(Obj)
  } else(return(paste0("No homoscedasticity, p-value = ",
                        (Lev["group","Pr(>F)"]))))
  if(Res["b", "Pr(>F)"] < 0.05){
    Post <- TukeyHSD(Obj)
    return(list(data.frame(Post$b) %>%

```

```

    rownames_to_column(var="b") %>%
    filter(p.adj<0.05),Res["b", "Pr(>F)"])))
  } else (return("null"))
}
#-----

```

In RStudio, create and open the .Rproject file for this project (at the root of the project), then open a new script and copy/paste the script above. Since the project is made of very few, simple functions, you can save everything in one script labelled *FUNCTIONS.R* to make it simple. But in a larger project, it is recommended to **properly split the sets of functions in different scripts**.

Now your script is ready and properly saved in the **R** folder of the project, you can use the *targets* package.

In the RStudio console, run the following: `install.packages("targets")` (only once, it is to install the package on your machine), then `targets::tar_config_set(script = "Reproducibility_targets.R", store = "Results", project = "Reproducibility")` (only once, to set the basic configuration of your project). In the latter function, *script* defines the main dashboard script of the project, *store* defines the folder where you want to store the outputs and *project* defines the root folder of the project. This creates a *__targets.yaml* configuration file which is automatically stored at the root.

You can now start to use *targets*, running `use_targets(script = "Reproducibility_targets.R")` in the RStudio console. It creates the following script, labeled with the project's name, i.e. *Reproducibility_targets.R*. **Save it at the root of the project**, this is the dashboard of the workflow. It is always through it that you will run the workflow after having edited existing scripts or created new ones.

Let's see how *Reproducibility_targets.R* works:

- First, introduce a function to **clean your working environment**:

```
rm(list=ls())
```

- Then **load the packages needed for the workflow**. In our case, they are few:

```
tar_option_set(
  packages = c("readr", "dplyr", "ggplot2", "tibble", "car"))
```

- Now you need to **load the scripts of the workflow** (in the right order!). Here, we gathered all the functions in one script, which is much more simple:

```
tar_source("R/FUNCTIONS.R")
```

- Here is the tricky part: you have to list exhaustively and without error all the **functions**, their **arguments** and their **output**. The basic function to do so can be summarized as follow : `tar_target(OUTPUT, FONCTION(ARGUMENT))`

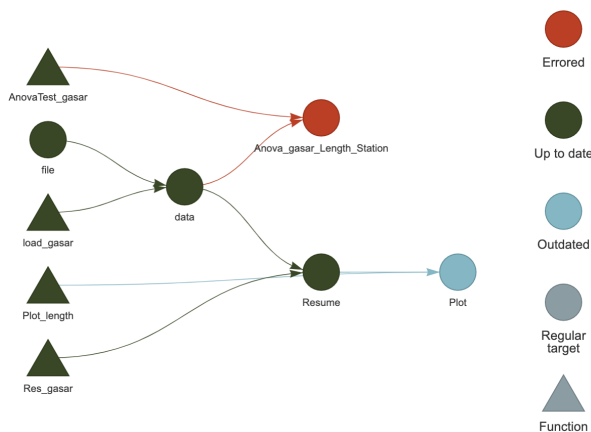
```
list(
  tar_target(file, "Data/Allo.csv", format="file"),
  tar_target(data, load_gasar(file)),
  tar_target(Resume, Res_gasar(data)),
  tar_target(Plot, Plot_length(Resume)),
  tar_target(Anova_gasar_Length_Station, AnovaTest_gasar(data, data$Longueurs,
                                                         data$Station)))
```

Now, your project is ready to be managed by *targets*! This script should contain only target definitions. Obviously, you can update it with new scripts, new functions, ne packages, etc... As your running through the project.

Several basic functions are available to interact with the workflow. Use them in the console, not in the script.

```
Sys.setenv(TAR_PROJECT = "Reproducibility") # Use it only when you open the
#workflow, it specifies which _yaml file to use to use the good configuration.
tar_manifest(fields = command) # Check if there is any error in the workflow
tar_visnetwork() # To see project's architecture (need the "visNetwork" package)
tar_make() # To run the workflow
tar_read() # To see the outputs
```

`tar_visnetwork()` is very useful, it visualizes the whole project in a dynamic plot. In our example, if you run it after `tar_make()`, the resulting figure will be:



Here you can see the different objects of the project and their state: outdated/up to date/errored. Consequently, you can have a clear idea, at any time, of the overall state of your project. For example, here, the target `Anova_gasar_Length_Station` is errored, meaning that there is something wrong in the function `AnovaTest_gasar()`.

Besides, `Sys.setenv()` is useful when your working directory contains several sub-projects. The specified argument relates to the content of the `__PROJECT.yaml` file where all the sub-projects of the repository are listed and where you define the folders in which store the related *outputs*.

4 Project implementation - the *renv* package for a transportable project-specific library

The *renv* package presents many interests to enhance the reproducibility of your projects:

- (i) It enables to **isolate** each project from each other by providing their own **libraries** (set of packages). Thus, you will never impact other projects by installing or updating packages within the current project, reducing dependencies issues.
- (ii) It facilitates the **transfer** of projects from one machine to another, by automating the installation of packages in their versions required for the project.
- (iii) It improves **reproducibility** by ensuring the use of the same library whatever the machine, the year, the user.

Semantic clarification: The term “*library*” refers to a folder containing all the *packages* used. Usually, with the function `library()` one load packages from the *system library* shared by all the R projects. With *renv*, *specific libraries* are created for each project.

Once you have installed the package (`install.packages("renv")`), you can run `renv::init()` to convert your project. Five entities will be created at the root of the project:

- The **project’s library**: *renv/library*. Here are stored all the packages.
- *renv/activate.R*: contains some informations to inform *renv* about how to initiate the project and retrieve packages listed in *renv.lock*.
- The **project’s settings**: *renv/settings.dcf*. It contains some *renv* configuration settings.
- The **lock file**: *renv.lock*. It contains all metadata needed to re-install the packages on other computers.
- The **R profile**: *.Rprofile*. It is used by *renv* to configure each new R session so it can use the project library.

As you progress through the project, you will probably need to install, load and update packages. To properly update the *renv.lock* file, you just have to run `renv::snapshot()`.

Finally, to properly share your project’s library, send with your project the following files : *renv.lock*, *.Rprofile*, *renv/.gitignore*, *renv/setting.dcf*, *renv/activate.R*. Then, when the project will be open on another machine, just executing `renv::restore()` will automatically re-install

the whole project's library from the *renv.lock*. It is possible that some dependencies require a manual installation but *renv* usually drives you well in this process.

Here are the four function `init()`, `snapshot()`, `status()`, `restore()`, enabling the basic use of *renv* to enhance your project reproducibility. If you are interesting in more advanced use of *renv*, you can find more informations [here](#). A great feedback by Elise Maigne is also available [here](#).

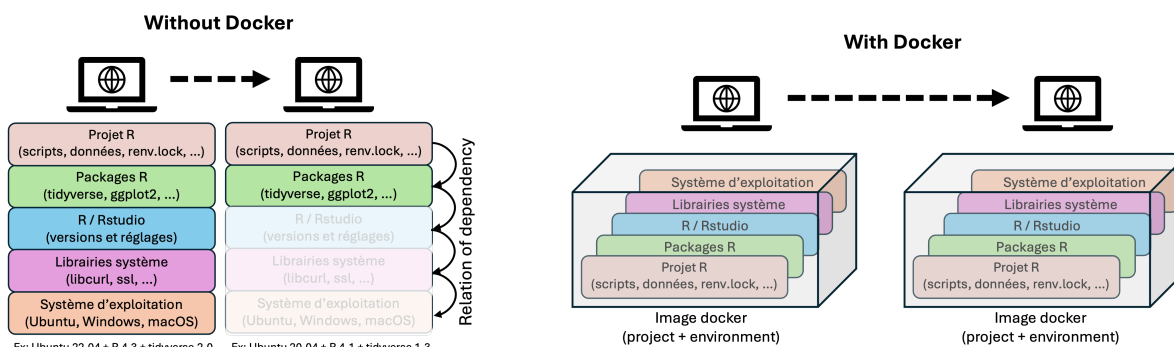
5 Making your project fully reproducible with the power of images & containers - *Docker*

5.1 What images & containers are?

Consider your current computational research project (using R/Rstudio). It is made of several tied layers:

- **The R project:** scripts, data, `renv.lock`, ...
- **The project's library:** *tidyverse*, *ggplot2*, *DESeq2*, *dplyr*, ...
- **R (software) and RStudio (IDE):** with specific versions and settings
- **The system libraries:** *libcurl*, *ssl*, ... required for R, Rstudio and R packages to be executed.
- **The Operating System:** Ubuntu, Windows, macOS, ...

When you share your project, you basically share the first layer, or the two first if you use *renv*. But the three others are of critical importance to ensure the project to be reproducible - even more when you are working with some specific tools that may require specific system libraries, etc. *Docker* helps you to solve this.



Basically, *Docker* works with **images** that are isolated copies of projects and the whole related environment, i.e. the above 5 points. These images can be stored on dedicated clouds,

shared, and run on any machine independently from the local environment. This is powerful and supposedly secures the reproducibility of the project throughout time.

From there, a **container** is nothing but an isolated image being executed. Hence, from a unique image, you can create several containers being executed in parallel.

You can easily install *Docker* on your machine following this [link](#).

5.2 Creating a *Docker* image - the Docker file

The *Dockerfile* is a recipe, *the formal description of what your image contains and what actions to do when it is executed*. Before going further in describing the Dockerfile and building a minimal image example, let's have a look at it.

```
#Start image from rocker/r-ver:4.5.2
FROM rocker/r-ver:4.5.2

# Create the project folder in the container
WORKDIR /project

# Update linux available libraries and install those necessary.
RUN apt-get update && apt-get install -y \
    curl \
    pandoc \
    libcurl4-openssl-dev \
    libfreetype6-dev \
    libpng-dev \
    libtiff5-dev \
    libjpeg-dev \
    libwebp-dev \
    libcairo2-dev \
    libssl-dev \
    libxml2-dev \
    libfontconfig1-dev \
    libharfbuzz-dev \
    librrlibid1-dev \
    libgsl-dev \
    && rm -rf /var/lib/apt/lists/*

# Install Quarto CLI
ARG QUARTO_VERSION=1.10.18

RUN curl -Ls "https://github.com/quarto-dev/quarto-cli/releases/download/v${QUARTO_VERSION}/quarto-${QUARTO_VERSION}-linux-amd64.tar.gz" -o quarto.tar.gz \
    && mkdir -p /opt/quarto \
    && tar -xzf quarto.tar.gz -C /opt/quarto --strip-components=1 \
    && ln -s /opt/quarto/bin/quarto /usr/local/bin/quarto \
    && rm quarto.tar.gz

# During image building, run R and execute the following code.
RUN R -e "install.packages('renv', repos='https://cloud.r-project.org')"

# Copy files needed for rend::restore() from local project to the container
COPY reny.lock reny.lock
COPY .Rprofile .Rprofile
COPY reny/activate.R reny/activate.R
COPY reny/settings.json reny/settings.json

# During image building, run R and execute the following code : restore the project's reny library
RUN R -e "renv::restore()"

# Once environment is ok, copy the full project
COPY . .

# Specify where targets is supposed to find its script
ENV TAR_PROJECT=reproducibility
```

The Docker file is actually built on few main commands:

- **FROM:** to define a pre-existing image from which to build your own;
- **RUN:** to execute a specific command during image building. It modifies the image during its building;

- **COPY:** to copy objects (files or folders) from your local environment to the image during its building;
- **CMD:** to execute a specific command while starting a container from the image.

Then, the image is built following an iterative process, i.e. every step is validated before going on subsequent steps.

1. Define the source image from which to build your own.

In this example : *rocker/r-ver:4.5.2* is an image containing all things necessary to use R v4.5.2 and RStudio. You can find the image and all its versions [here](#).

FROM rocker/r-ver:4.5.2

2. Define the working directory of the project in the image.

WORKDIR /project

→ **Here, the objective is nothing but having a Linux environment with the good R version.** Build the library to check if there is no error - in your local project: `docker build -t projectName .`

3. Rebuild the renv project's library using *renv*.

We first have to install *renv* into the image...

RUN R -e "install.packages('renv', repos='https://cloud.r-project.org')"

...and copy from the local project all files needed to restore the project's library...

COPY renv.lock renv.lock

COPY .Rprofile .Rprofile

COPY renv/activate.R renv/activate.R

COPY renv/settings.json renv/settings.json

...then perform project's library restoration:

RUN R -e "renv::restore()"

-> This step is iterative. First tries will identify system dependencies required to some R packages. If one package requires a specific system library, we add it to the image until the installation of all R packages is successfully completed. Each try consist in building the image:
`docker build -t projectName ..`

To add Linux dependencies to the image:

`RUN apt-get update && apt-get install -y` followed by the name of the libraries (see above example).

4. Once R environment is ok, add any external tool.

For instance, if your pipeline uses QUARTO CLI to render the work, you must install it in your image:

```
ARG QUARTO_VERSION=1.10.18
```

```
RUN curl -Ls "https://github.com/quarto-dev/quarto-cli/releases/download/v${QUARTO_VERSION}/${QUARTO_VERSION}-linux-amd64.tar.gz" -o quarto.tar.gz \
```

```
&& mkdir -p /opt/quarto \
```

```
&& tar -xzf quarto.tar.gz -C /opt/quarto --strip-components=1 \
```

```
&& ln -s /opt/quarto/bin/quarto /usr/local/bin/quarto \
```

```
&& rm quarto.tar.gz
```

Execute in local: `docker build -t projectName .` to build the image with the QUARTO CLI

5. Once the whole environment of your project is ok (OS, R, Rstudio, system libraries, R packages, QUARTO CLI), add the whole project to the image.

`COPY . .` = it copies the whole content from your local root project to the working directory of the image.

6. Tell *targets* where to find its script, then run it.

```
ENV TAR_PROJECT=reproducibility
```

CMD ["R", "-e", "targets::tar_make()"] if you want the pipeline to be executed and results produced each time you start a container

RUN R -e "targets::tar_make()" if you want the pipeline to be executed once during the image building. (Both option can be applied together.)

5.3 Looking at the *Docker* image, run it and retrieve the results

Well done, if you arrived here it means that your first *Docker* image has been built successfully ! Now, you may want to interact with your *Docker* image. For instance, you may want to **enter and navigate** into it. To do so, you must create a *container* (executing image): `docker run --rm -it projectName bash`. If you put `RUN R -e "targets::tar_make()"` at the end of the Dockerfile, then you will find the results stored in the image. But if not, you won't find the results.

Probably the best way to execute a container and retrieve the outputs is to proceed as follow:

- `docker run --name projectName_run projectName` - here you run the container at give him an identifier (i.e. *projectName_run*);
- `docker cp projectName_run:/project ./projectName_output` - you copy the content of the container into your local environment under the label *projectName_output*;
- `docker rm projectName_run` - remove the container.

Following this process, you will retrieve in the local *./projectName_output* folder the whole project from the image with the outputs.

Another way to proceed is to build volumes. In other words, you start a container and you link each container folder to a local folder. Hence, any file written in the targeted container folder will be written in the related local folder at the same time. The command is quite simple and has this structure: `docker run -v host:container projectName`. For instance:

- `docker run --rm \`
- `-v $(pwd)/resultsDocker:/project/results \`
- `-v $(pwd)/reportDocker/article:/project/report/article \`
- `projectName`

Here, *resultsDocker* and *reportDocker/article* are the exact local copies of */project/results* and *project/report/article* from the container. Although the `--rm` argument remove the container after it is executed, the local copies will stay on your machine.

Finally, if you encapsulate a project within a Docker image, anyone can re-execute a container from the image and retrieve locally the exact same outputs as yours. This is powerful!

5.4 Sharing your project as *Docker* image

There are at least two options to share your project as a *Docker* image.

The first one consists in pushing the Dockerfile to the GitHub repository of your project with the source code. Basically, if you already share your R projects on GitHub (or GitLab), you just have to add your Dockerfile. Doing so, someone who wants to reproduce your results would just have to run the following commands:

```
git clone <your_repository URL> to download the project from GitHub
cd projectName to enter the project
docker build -t projectName . To rebuild the image
docker run --rm \
-v $(pwd)/resultsDocker:/project/results \
-v $(pwd)/reportDocker/article:/project/report/article projectName to execute
the container and retrieve the output in local.
```

A second option is to publish the built image on a dedicated platform. Both options are not mutually exclusive and, if you use GitHub to share your all project, you can publish the built image on GitHub Container Registry; it is free and linked to your GitHub repository. To do so, run the following commands:

```
docker build -t ghcr.io/<username>/projectName:1.0 .
docker push ghcr.io/<username>/projectName:1.0
```

After that, your built image is available for anyone. It is also linked to your project GitHub repository as a package. If someone want to retrieve it, just run:

```
docker pull ghcr.io/<username>/projectName:1.0
docker run --rm -v $(pwd)/resultsDocker:/project/results ghcr.io/<username>/projectName:1.0
```

6 Project versioning - Git and Github

6.1 What are we talking about?

Git is a **version control** system initially designed for **collaborative work** in the informatics community. Therefore, it enables to follow the evolution of all the documents within a project - a **repository**.

Warning: **Git** can be hard to handle. It is all but intuitive... If you don't work in a *collaborative mode*, there probably are plenty of easier ways to ensure local versioning of your project. In other words, dive into the world of Git only makes sense if you work on projects with colleagues susceptible to participate in the scripts, manuscripts and other outputs of the project (it is actually the case for almost all the researchers!). In that case, you will not only need Git but its *remote* counterpart. There are different ones but the most commonly used is **GitHub**.

In the following, you will learn how to **use RStudio in version control with Git and GitHub**. You need to have a free account on [GitHub](#).

This section is a synthesis from the amazing *Happy Git and Github for the useR*. It aims at providing **clear and simple** guide to start with Git and GitHub. But, as all simple things, it is mainly incomplete. If you want to go further into *versioning*, do not hesitate to look at *Happy Git and Github for the useR*.

6.2 Installing Git

There are several ways to install Git but they are not all equivalent.

- On a **windows** machine, it is advised to follow this [protocol](#) which automatically installs the **Git bash** and some other required tools. Besides, it stores the .exe file in a conventional folder easy to use for RStudio.

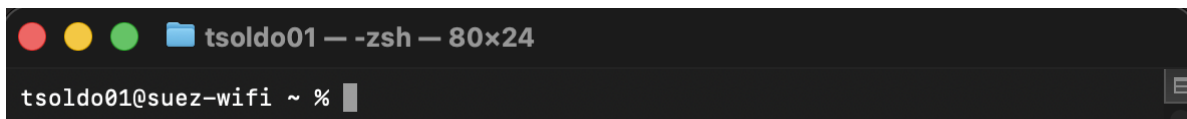
- On a **macOS** machine, the best way is to directly open the command line and run this two commands: `git --version`, then `git config`. A pop-up should appear, proposing the installation. Accept it!

6.3 Configuring Git

Before going to R, you have to be introduced with the basic Git commands. On windows, open the **Git Bash** (accessible from the “Search” menu of your machine); on macOS, just open the **terminal**.

Once open, these apps are nothing but a **shell**, a user interface enabling to understand commands sent in informatic language. We are going to use the **bash** language. A comprehensive list of basic bash commands are available [here](#).

Here is your interface, the dollar symbolises the start of the command line:



From there, you can navigate in your machine and perform some basic actions:

```
cd PATH          # Navigate toward the specified directory;
ls -l            # Display the content of the directory;
touch NAME        # Create a new file;
rm NAME          # Delete a file;
mkdir NAME        # Create a new folder;
rm -r NAME        # remove a folder
```

Once you master these functions, it's time to **configure** your Git!

The Git synthaxe is always `Git COMMANDE --ARGUMENT`. To configure Git, we use `git config`:

```
git config --global user.name "your name"
git config --global user.email "your email"
git config --global core.editor "emacs" #or any other preferred editor
git config --global init.defaultBranch "master" #Main branch of your projects
```

Numerous other config options are available, that you have to discover by yourself, for instance [here](#). To see your complete set of options:

```
git config --global --list
```

6.4 Connecting Git and GitHub

Now Git knows you, let's introduce him your GitHub! Here as well, several ways to proceed exist. I will just present the easier.

First, you must generate a **PAT** (Personal Access Token). The PAT is a kind of password testifying that you belong to the Git and defining what you can do or not.

Two ways to generate a **PAT**:

- Directly *via* Github, once connected, click on *settings>Developer settings>Personal access tokens>Generate new token*
- *Via* RStudio, install the *usethis* packages (`install.packages("usethis")`) and run the following function `usethis::create_github_token()`. A pop-up will open to (i) **describe** the PAT (its purpose), (ii) **define an expiration date** (advised by GitHub) and (iii) **define a scope**. Once you specify these params, click *Generate token*.

BEFORE TO CLOSE THE POP-UP make sure to properly **copy** the **PAT**, cause once you close it you will no be able to access it again! I advise you to copy the PAT in a safe place, easily accessible by RStudio, Git and GitHub using `gitcreds::gitcreds_set()`. R will ask you to give your PAT to store it in a safe place:

```
gitcreds::gitcreds_set()

? Enter password or token: ghp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
-> Adding new credentials...
-> Removing credentials from cache...
-> Done.
```

Remark: Since your PAT is temporary, you will have to regularly repeat the protocol. It is not practical but it is for a security purpose.

Here we are! You can now connect Git and GitHub.

- In Github, click the big green button *new*, just next to **repositories**. Fill the form and let the default parameters. Give your repository the same name as your local project you want to connect - i.e. *Reproducibility*, then click *Create repository*.
- In *Quick setup* copy the **HTTPS URL**.
- In your local command line, go to your *Reproducibility* folder, which is the root of your project containing the *.Rproject*, *targets* objects, *renv* library, etc.

- Start tracking versions :

```
git init
```

- Tell Git to consider the files in the folder (except the raw data that are an untouchable treasure):

```
git add FileName #To do it individually
```

```
# Else :
```

```
git add --all #To integrate all the files/folders in the version tracking, then  
git rm -r --cached Data/ #To remove the Data folder from the tracking.
```

- Make your first commit (we talk about that soon):

```
git commit -m "First commit"
```

- Run the following function with the HTTPS URL you have copied:

```
git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPOSITORY.git
```

- Your Github repository is now connected to your local project but it is still empty. To transfer the project to the GitHub repository:

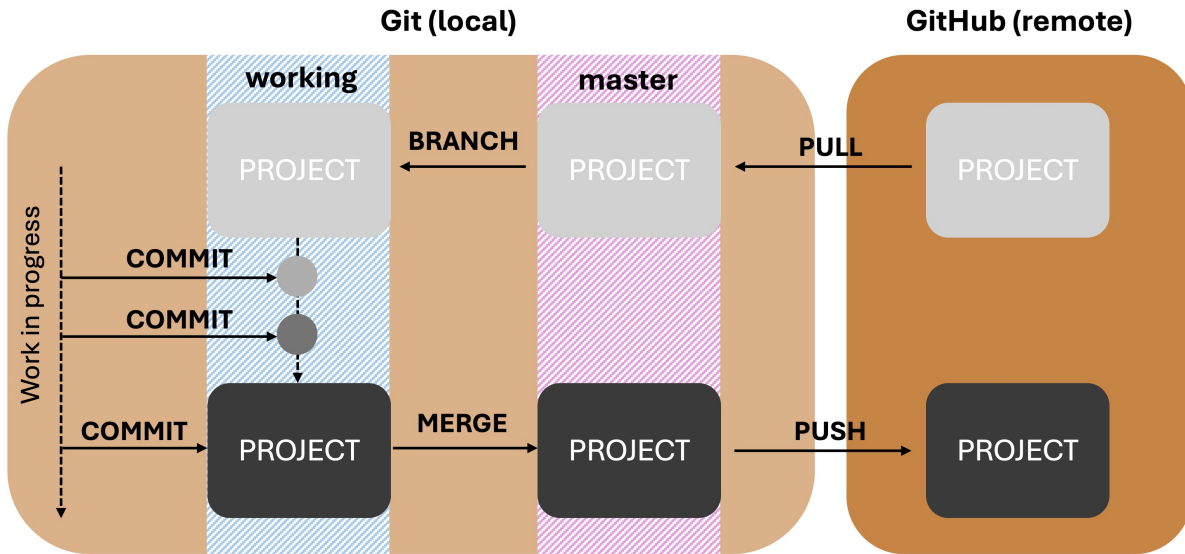
```
git push --set-upstream origin master
```

Refreshing the GitHub webpage, you should see your project. Now, consider the GitHub repository as the main localization of your project (the one that must always be up-to-date). Your local machine became a simple way to act on your project.

Remark: Using `Git clone URL HTTPS` you can do the opposite, that is import a repository from GitHub to your local machine.

6.5 Example for routine use

Now, you have to know the architecture of Git and GitHub, and some basic functions to start using it properly. A basic routine use of Git and GitHub is displayed in the following scheme:



Considering **the last version of your project is on GitHub**, start your working day by navigating to your local project's root using `cd PATH`. Then use `git pull` to load the last version of the project from GitHub to your local machine. You now have your up-to-date project in the *master* branch of Git.

To secure your work, I advise you to immediately create a parallel branch on which you can do all the modifications you want before merging them to the main branch. Always give **explicit names** to the different branches you will create. Here, for instance: `git branch working`. Once the branch exists, you have to position yourself on it using `git checkout working`. If you are lost and want to know on which branch you are, use `git branch`.

Ok, now you are on the parallel branch (*working*), which contains the same version of the project as the main branch (*master*). You can start to work!

For instance, you can create the `README.txt` file at the root of the project or modify it if it already exists. As long as Git doesn't track the file, this modification is on your computer. You have to run

- `git add FileName` to ask Git to track the modification. If you run `git status`, the tracked files are presented in green.

+`git commit -m "A message to describe the commit`, to ask Git to capture the modification. The **commit** is a kind of snapshot of the project. A comment is mandatory to perform a commit, so you can easily know what was committed if you go back to it later. It is advised to commit consistent set of objects independantly.

Now, the modification belongs to your *working* branch only. If you change to the *master* branch, the README.txt file disappears. Now, go back to the *working* branch: the README.txt file reappears! It is magic...

Remark : If you are lost, you can check the Git status of the project with: `git status`.

Thus, after each commit, it is a new version of your project that is saved on the branch. Don't need to commit after each modification. Do it at strategic steps, when a significant modification has been added: new script, new data, new result... Committing several times along the working day, the project evolves and keeps a trace of each key step: the project versions. Here is the interest of versionning, in case of error, or any problem, you can go back to the last commit using `git reset HEAD^`.

At the end of the work session, you can transfer the last version toward GitHub. Two possibilities:

- (i) You are satisfied of the work, confident in the modifications, and nobody needs to validate them. Thus, go to your *master* branch: `git checkout master`, then merge the *working* branch to the *master*: `git merge working`. Now run `git push` to make the transfer toward GitHub.
- (ii) Your modifications must be checked by a colleague before integrating them to the project. Directly push the *working* branch to GitHub using `git push` directly from that branch.

Now, refreshing the GitHub page, you can see the new version of the project.

Remark: Thanks to the commits, you can move into the history of the project. Use `git log` to access the history:

```
Reproducibility — less • git log — 105x24
tsoldo01@suez-wifi Reproducibility % git log
commit 05c92d014574be0ea18c4b8d889cc0db1281d356 (HEAD -> master, origin/master, origin/HEAD)
Author: Thomas Sol Dourdin <t.soldourdin@gmail.com>
Date: Thu Oct 23 16:11:40 2025 +0200

    update.

commit 2c806b1604504e76d5ebba2b48b5bf4ab32968d6 (working_branch)
Author: Thomas Sol Dourdin <t.soldourdin@gmail.com>
Date: Thu Oct 23 12:42:09 2025 +0200

    cleaning

commit 20a0ba908461626448b50590a5787637fee04a73
Author: Thomas Sol Dourdin <t.soldourdin@gmail.com>
Date: Thu Mar 6 10:00:40 2025 +0100

    blabla
```

Each commit has an identification number - the **hash**, an **author**, a **creation data** and a **message**. Consequently, you can find any version of the project and go back to it if necessary **without modifying nor deleting the subsequent version**. This is done with `git checkout CommitHash`. You can then see the related project version, make some modifications or even create a new branch from this particular commit (`git checkout -b NewBranch CommitHash` or `git switch -c <NewBranch>` if you are already on the commit) to start some parallel work you can merge later.

The use of Git and GitHub can be much more subtle and complex than what is present above. To learn more on Git function and improve your versioning skills, go [there!](#)

6.6 Connect RStudio to Git and GitHub

If you regularly use RStudio, you can simplify your interaction with Git and GitHub from the IDE. Let's connect RStudio to Git and GitHub!

There are some prerequisites:

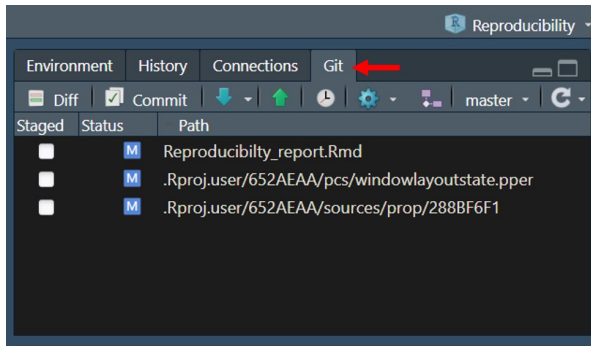
- Have a Github free account;
- Have a recent version of R and RStudio on your machine;
- To be introduced to basics of Git;
- Have a valid connexion between your Git and GitHub.

If you did not restart RStudio since you installed Git, do it.

Now, be sure RStudio properly localize the Git .exe. If you followed the **section 5.2** it should be ok.

To be sure, click on *Tools > Global options... > Git/SVN* in RStudio. In this menu, check if the path to Git .exe is correct. Then check “*Enable version control interface for Rstudio projects*” and click on *Apply* at the bottom of the window. Restart Rstudio and open the *Reproducibility* project whose already tracked by Git.

You should see at the top-right of the window a button *Git* enabling to **track the modifications**, make **commit**, **push**, **pull**, create **new branches** and navigate between them.



The use of Git *via* RStudio will not be described here. A good tuto can be find [there](#).

7 Rendering the work

7.1 Introduction to Rmarkdown

Here, we will only speak about the written reporting of the project: articles, reports, books, etc.

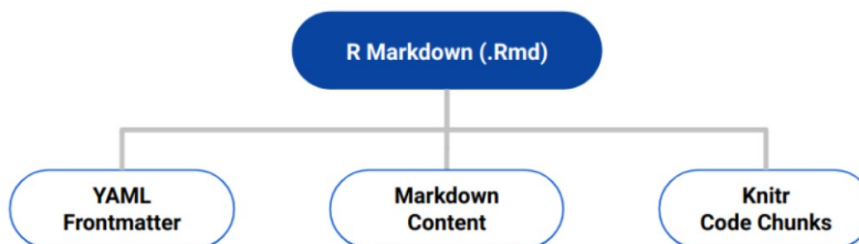
Ordinary people usually use softwares like **Word**, **OpenOffice**, **LibreOffice**, etc. These text edit softwares are defined as **WYSIWYG**: “What You See Is What You Get”. Their main advantage is the click-button interface which are very user friendly. Nevertheless, if you want to program a quite complex document with specific formatting, images, tables, or even code, the **WYSIWYG** softwares become very labrious.

Hopefully, there is an alternative! These are **WYSIWYM** softwares: “What You See Is What You Mean”. They are base on a **mark-up language** enabling to completely separete content and form. Some mark-up languages are well-know, i.e. **LaTeX** ou **HTML**. The principle is simple: by integrating *in the text* the marks of a particular form, the operator focuses only on the content. The formatting is automatically made by compilers.

The **WYSIWYM** associated to R is **Rmarkdown**, an adapted version from the original, very simple **Markdown**. Rmarkdown documents are dynamics. It means that they integrate both text - to convey a message - and code - to perform analyses and figures.

7.2 Structure of a Rmarkdown document

A Rmarkdown document if made of three main elements:



- **The frontmatter:** At the beginning of the doc, it is delimited with 3 dashes (—):

```

---
title: "Reproducibility"
author: "Tsoldour"
date: "`r Sys.Date()`"
output: pdf_document
toc: TRUE
---

```

By default, the frontmatter contains the metadata: title, author, date, output type. You can specify many other informations to **customise** your doc. For instance here: `toc: TRUE` activate an automatic table of content at the beginning of the doc.

- **The content:** The content is nothing but the text you will write and customise with the **marks**.

```

488 | For instance, you can write in *italic* or in **bold**.
489
490 | You can also make some bullet points:
491
492 + first bullet;
493
494 + second bullet;
495
496 + third bullet
497
498 | Or event insert code directly in the text: `usethis::create_github_token()`
499

```

For instance, you can write in *italic* or in **bold**.

You can also make some bullet points:

- first bullet;
- second bullet;
- third bullet

Or event insert code directly in the text: `usethis::create_github_token()`

- **the code chunks:** Here is the Rmarkdown interest: you can insert **chunks**, that are spaces dedicated to R language used to produce analyses and figures directly in the document.

If you want to know (much) more about Rmarkdown, go to the [markdown guide](#) and [Rmarkdown guide](#).

Recently, the developers of Rstudio - Posit - published a new version of Rmarkdown: **quarto**. It is mostly the same, with the structure frontmatter, content and chunks, and using markdown. But it has the advantage of accepting R and python languages, and to facilitate the creation of books and websites. If you know how to use Rmarkdown, you won't have any difficulties to adopt quarto ! To download, it is [here](#) and the man is [here](#).

What is very interesting with Rmarkdown and quarto is that they can be used in association with a *targets* project. It is very simple, you just have to install and load in the `__targets.R` script the *tarchetypes* package. Then, at the end of the list of targets, just add `tar_render(Name, path_to_the_Rmarkdown)` or `tar_quarto(Name, path_to_the_quarto)`. As for the other targets, use `tar_make()` to run it and your dynamic report will be automatically outputted. In your Rmarkdown or quarto document, you can point toward targets to directly include the outputs of your workflow in your report/article/book, etc. In particular to write scientific articles, it is powerful! A quarto example based on this reproducibility project is available here. Check the files [Article.qmd](#) and [Reproducibility_targets.R](#) at the root of the project.

8 Summary: common reproducibility anti-patterns (and how to avoid them)

Even with the best intentions, some common habits strongly undermine reproducibility. Below is a non-exhaustive list of frequent anti-patterns, along with recommended alternatives.

- **Using `setwd()` in scripts**

Why it is a problem:

Absolute paths are machine-specific and will break when the project is moved or shared.

Prefer: Work inside an RStudio project and rely exclusively on relative paths.

- **Modifying raw data files**

Why it is a problem: Raw data are the foundation of the entire analysis. Any modification makes results unverifiable.

Prefer: Treat raw data as read-only and perform all transformations through scripts.

- **Copy-pasting results into Word or PowerPoint**

Why it is a problem: Manual transfer introduces errors and disconnects results from the code that produced them.

Prefer:

Use R Markdown or Quarto to generate figures, tables, and text directly from the analysis.

- **Running analyses interactively without scripts**

Why it is a problem:

Point-and-click actions leave no trace and cannot be reproduced.

Prefer: Script every analytical step, even exploratory ones.

- **Installing or updating packages without tracking versions**

Why it is a problem: Results may change over time or across machines due to dependency updates.

Prefer: Use `renv` to lock package versions and ensure a stable computational environment.

- **Working without version control**

Why it is a problem:

Changes are irreversible, undocumented, and difficult to share or review.

Prefer:

Use Git to track all modifications to scripts, documentation, and configuration files.

Key idea

Reproducibility is not about perfection or rigidity.

It is about making your work **understandable, inspectable, and reusable** — by yourself and by others.

9 The reproducibility: a way to find help!

To produce analyses and scripts easily reproducible is also the best way to find help when needed. Whether you ask a colleague, a professor or even an artificial intelligence, you have more chance to find a satisfying answer if you give understandable and usable piece of code - a “*reprex*”: REPRoducible EXample.

A good reprex relies on few bases that must be considered seriously:

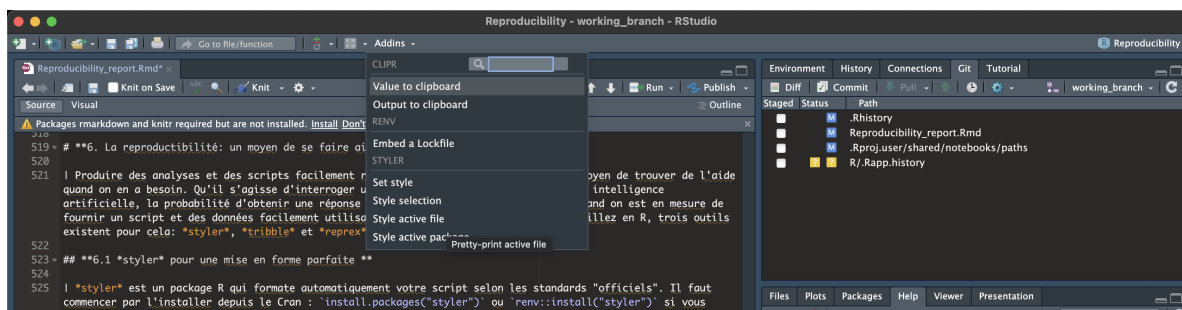
- The code and data must be as short as possible. Do not send your whole script, only the problematic part! In the same way, a data set shortened at the maximum is often enough to render its structure.
- The code must run properly with a perfect reproduction of the error. To do so, you must include all the call to required packages (`library()`) et used objects.
- The code must be easy to read.
- The code must be easy to copy and execute.

With R, there are three packages to guarantee all of this: *styler*, *datapasta* et *reprex*.

9.1 *styler* for a perfect formatting

styler is a R package that automatically formats your script based on “official” standards. First, install it: `install.packages("styler")`. Then, restart the R session.

Now you can use *style* via the Rstudio addin as bellow:



or via 3 main functions :

- `style_pkg()`: to format .R, .Rprofile, .Rmd, etc. files
- `style_file()`: to format any type of specified file.
- `style_text()` : to format text specified as vector.

The most simple is to use the addin. For instance, using `style selection` after having selected this code:

```
a<-c(1:10 )
b <- rep(c("A","B"), by= 5)
data <-data.frame(a , b)%>%
mutate(c= ifelse(b="A", "avocat", "banane"))
```

We obtain:

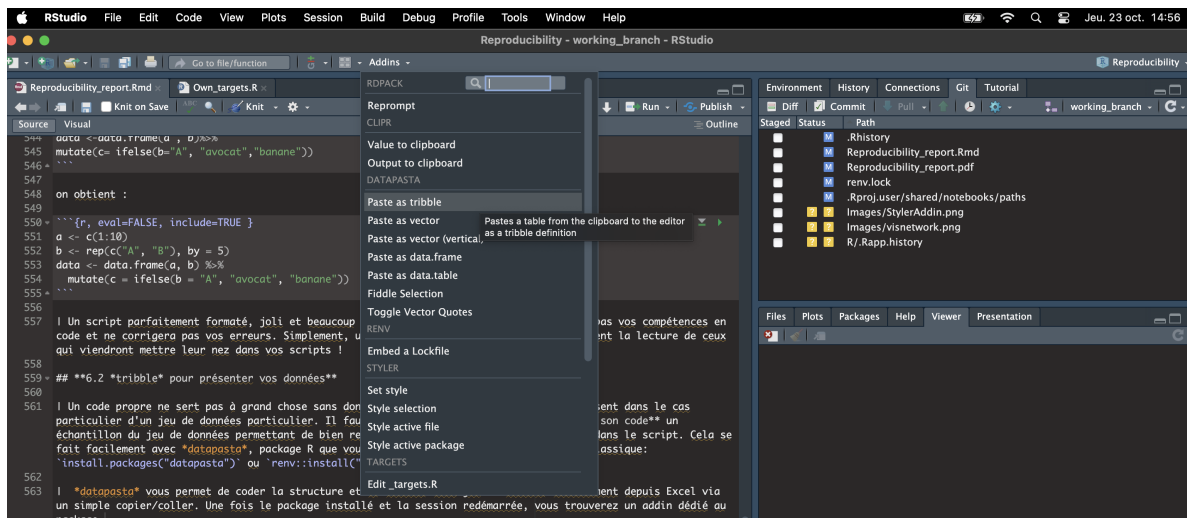
```
a <- c(1:10)
b <- rep(c("A", "B"), by = 5)
data <- data.frame(a, b) %>%
  mutate(c = ifelse(b = "A", "avocat", "banane"))
```

A script perfectly formatted and clear. *styler* will not correct the code errors, nor enhance your coding skills. But a standardize formatting simplifies the reading for those who will check your scripts.

9.2 *tribble* to present your data

A clean code is not useful without data, and the vast majority of issues happens in the particular case of a particular data set. Thus, you must put **into your code** a sample of the related dataset, enabling to properly reproduce the actions coded in the script. You can do it with the *datapasta* R package. Install it with the classical protocol: `install.packages("datapasta")`.

datapasta enables to code the structure and the content of a dataset directly from any spreadsheet software, *via* a simple copy/paste action. Once the package is installed, and your session has restarted, a dedicated addin appears:



Then, to produce a simple example of your data, you just have to:

- open the data in a spreadsheet et copy a sufficient fraction in your clipboard:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
	Station	Espece	Lot	Longueurs	PSI	IC															
1	Missirah	Crassotrea_gasar	1	6,5	0,9993	0,15373846															
2	Missirah	Crassotrea_gasar	1	5,2	0,9993	0,19217308															
3	Missirah	Crassotrea_gasar	1	5,5	0,9993	0,18169091															
4	Missirah	Crassotrea_gasar	1	5,9	0,9993	0,16937268															
5	Missirah	Crassotrea_gasar	2	7,1	1,1401	0,16057746															
6	Missirah	Crassotrea_gasar	2	6,9	1,1401	0,16523188															
7	Missirah	Crassotrea_gasar	2	5	1,1401	0,22802															
8	Missirah	Crassotrea_gasar	2	4,9	1,1401	0,23267347															
9	Missirah	Crassotrea_gasar	3	6,4	0,954525	0,14914453															
10	Missirah	Crassotrea_gasar	3	6,1	0,954525	0,15647951															
11	Missirah	Crassotrea_gasar	3	6,7	0,954525	0,14246642															
12	Missirah	Crassotrea_gasar	3	5,8	0,954525	0,16457328															
13	Missirah	Anadara_senilis	1	3,3	0,53494	0,16210303															
14	Missirah	Anadara_senilis	1	3,5	0,53494	0,15284															
15	Missirah	Anadara_senilis	1	3,3	0,53494	0,16210303															
16	Missirah	Anadara_senilis	3	3	0,53494	0,17831333															
17	Missirah	Anadara_senilis	1	3,3	0,53494	0,16210303															
18	Missirah	Anadara_senilis	2	3,7	0,47298	0,12783243															
19	Missirah	Anadara_senilis	2	2,8	0,47298	0,16892143															
20	Missirah	Anadara_senilis	2	3,2	0,47298	0,14780625															
21	Missirah	Anadara_senilis	2	3,2	0,47298	0,14780625															
22	Missirah	Anadara_senilis	2	3,4	0,47298	0,13911176															
23	Missirah	Anadara_senilis	2	3,4	0,47298	0,13911176															

- in Rstudio, click on “*paste as tribble*”, or run `datapasta::tribble_paste()`: the code that reproduces your data is automatically inserted in your script:

```
datapasta::tribble_paste()
df <- tibble::tribble(
  ~Station, ~Espece, ~Lot, ~Longueurs, ~PSI, ~IC,
  "Missirah", "Crassotrea_gasar", 1L, 65, "0,9993", "0,153738462",
  "Missirah", "Crassotrea_gasar", 1L, 52, "0,9993", "0,192173077",
  "Missirah", "Crassotrea_gasar", 1L, 55, "0,9993", "0,181690909",
```

```

"Missirah", "Crassotrea_gasar", 1L, 59, "0,9993", "0,169372881",
"Missirah", "Crassotrea_gasar", 2L, 71, "1,1401", "0,160577465",
"Missirah", "Crassotrea_gasar", 2L, 69, "1,1401", "0,165231884",
"Missirah", "Crassotrea_gasar", 2L, 5, "1,1401", "0,22802",
"Missirah", "Crassotrea_gasar", 2L, 49, "1,1401", "0,232673469",
"Missirah", "Crassotrea_gasar", 3L, 64, "0,954525", "0,149144531",
"Missirah", "Crassotrea_gasar", 3L, 61, "0,954525", "0,156479508",
"Missirah", "Crassotrea_gasar", 3L, 67, "0,954525", "0,142466418",
"Missirah", "Crassotrea_gasar", 3L, 58, "0,954525", "0,164573276",
"Missirah", "Anadara_senilis", 1L, 33, "0,53494", "0,16210303",
"Missirah", "Anadara_senilis", 1L, 35, "0,53494", "0,15284",
"Missirah", "Anadara_senilis", 1L, 33, "0,53494", "0,16210303",
"Missirah", "Anadara_senilis", 1L, 3, "0,53494", "0,178313333",
"Missirah", "Anadara_senilis", 1L, 33, "0,53494", "0,16210303",
"Missirah", "Anadara_senilis", 2L, 37, "0,47298", "0,127832432",
"Missirah", "Anadara_senilis", 2L, 28, "0,47298", "0,168921429",
"Missirah", "Anadara_senilis", 2L, 32, "0,47298", "0,14780625",
"Missirah", "Anadara_senilis", 2L, 32, "0,47298", "0,14780625",
"Missirah", "Anadara_senilis", 2L, 34, "0,47298", "0,139111765"
)

```

If your data are already loaded in R and formatted as a dataframe, you can produce the same code with the following: `datapasta::dpasta(df)`.

Combining *styler* and *datapasta*, you are now able to produce a correctly formatted script reproducing the data you want to share as example. Now, you have to export the script in a format easy to read and execute from a GitHub-like forum, or StackOverflow, or even from an email.

9.3 *reprex* : exporting a fully reproducible script

The *reprex* package does the job! Once you have your perfect script, for instance:

```

# Reprex example
library(tibble)
library(dplyr)
library(ggplot2)

# Raw data example
df <- tibble::tribble(
  ~Station, ~Espece, ~Lot, ~Longueurs, ~PSI, ~IC,

```

```

"Missirah", "Crassotrea_gasar", 1L, 65, "0,9993", "0,153738462",
"Missirah", "Crassotrea_gasar", 1L, 52, "0,9993", "0,192173077",
"Missirah", "Crassotrea_gasar", 1L, 55, "0,9993", "0,181690909",
"Missirah", "Crassotrea_gasar", 1L, 59, "0,9993", "0,169372881",
"Missirah", "Crassotrea_gasar", 2L, 71, "1,1401", "0,160577465",
"Missirah", "Crassotrea_gasar", 2L, 69, "1,1401", "0,165231884",
"Missirah", "Crassotrea_gasar", 2L, 5, "1,1401", "0,22802",
"Missirah", "Crassotrea_gasar", 2L, 49, "1,1401", "0,232673469",
"Missirah", "Crassotrea_gasar", 3L, 64, "0,954525", "0,149144531",
"Missirah", "Crassotrea_gasar", 3L, 61, "0,954525", "0,156479508",
"Missirah", "Crassotrea_gasar", 3L, 67, "0,954525", "0,142466418",
"Missirah", "Crassotrea_gasar", 3L, 58, "0,954525", "0,164573276",
"Missirah", "Anadara_senilis", 1L, 33, "0,53494", "0,16210303",
"Missirah", "Anadara_senilis", 1L, 35, "0,53494", "0,15284",
"Missirah", "Anadara_senilis", 1L, 33, "0,53494", "0,16210303",
"Missirah", "Anadara_senilis", 1L, 3, "0,53494", "0,178313333",
"Missirah", "Anadara_senilis", 1L, 33, "0,53494", "0,16210303",
"Missirah", "Anadara_senilis", 2L, 37, "0,47298", "0,127832432",
"Missirah", "Anadara_senilis", 2L, 28, "0,47298", "0,168921429",
"Missirah", "Anadara_senilis", 2L, 32, "0,47298", "0,14780625",
"Missirah", "Anadara_senilis", 2L, 32, "0,47298", "0,14780625",
"Missirah", "Anadara_senilis", 2L, 34, "0,47298", "0,139111765"
)

# Tidying data
df_OK <- df %>%
  select(Espece, Lot, Longueurs, IC) %>%
  mutate(Sampler = ifelse(Lot == 1L, "Xavier",
                           ifelse(Lot == 2L, "Romain",
                                   ifelse(Lot == 3L, "Bertrand", "NA")
                                )
  )
))

# Plotting length
df_OK %>%
  ggplot(aes(x = Espece, y = Longueurs, fill = Sampler)) +
  geom_boxplot() +
  theme_bw()

```

You just have to copy the script in the clipboard and run `reprex()` in the Rstudio console. The Viewer displays the rendering and the clipboard contains the code to paste in your message

(on GitHub, StackOverflow, email, ChatGPT, etc.).

In the following example, where a plot is integrated, the line `<!-- -->` (**do not modify!**) enables to visualize the plot without running the code. It can be very useful if your issue is related to the graphic!

To go further with `reprex()`, you have to know some arguments:

- `venue = c("gh", "so", "ds", "r", "rtf", "html")` must be set based on the destination of your example. From left to right, the values correspond to GitHub, stackOverflow, Rstudio community, “R-friendly” output, Rich-Text Format, HTML document.
- `si = FALSE` by default, but if you specify `TRUE`, then your session info, (e.g. package versions) are included in the reprex, which may be required in some cases.